

Exploratory Data Analysis (EDA) Guide

Stock Challenge - Week 1

This guide provides step-by-step instructions and code examples for performing comprehensive EDA on the financial news dataset (raw_analyst_ratings.csv).

Dataset Overview

File: data/Data/raw_analyst_ratings.csv

Size: ~1.4M+ rows

Columns:

- headline: Article headline text

1. Descriptive Statistics

1.1 Headline Length Analysis

Objective: Calculate basic statistics for headline text lengths.

Key Metrics to Calculate:

- Mean, median, standard deviation

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data (use chunksize for large files)
df = pd.read_csv('data/Data/raw_analyst_ratings.csv')

# Calculate headline lengths
df['headline_length'] = df['headline'].str.len()
df['headline_word_count'] = df['headline'].str.split().str.len()

# Basic statistics
print("Headline Length Statistics:")
print(df['headline_length'].describe())
print("\nHeadline Word Count Statistics:")
print(df['headline_word_count'].describe())

# Visualizations
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
```

import p

```
# Histogram of headline lengths
axes[0, 0].hist(df['headline_length'], bins=50, edgecolor='black')
axes[0, 0].set_title('Distribution of Headline Character Lengths')
axes[0, 0].set_xlabel('Character Count')
axes[0, 0].set_ylabel('Frequency')

# Box plot
axes[0, 1].boxplot(df['headline_length'])
axes[0, 1].set_title('Box Plot: Headline Lengths')
axes[0, 1].set_ylabel('Character Count')

# Word count distribution
axes[1, 0].hist(df['headline_word_count'], bins=30, edgecolor='black', color='orange')
axes[1, 0].set_title('Distribution of Headline Word Counts')
axes[1, 0].set_xlabel('Word Count')
axes[1, 0].set_ylabel('Frequency')

# Violin plot for word count
sns.violinplot(y=df['headline_word_count'], ax=axes[1, 1])
axes[1, 1].set_title('Violin Plot: Word Count Distribution')

plt.tight_layout()
plt.savefig('notebooks/headline_length_analysis.png', dpi=300, bbox_inches='tight')
plt.show()
```

Key Questions to Answer:

- What is the typical headline length?

1.2 Articles Per Publisher

Objective: Count articles per publisher to identify most active publishers.

Implementation Steps:

```
# Count articles per publisher
publisher_counts = df['publisher'].value_counts()

# Top 20 most active publishers
top_publishers = publisher_counts.head(20)

print("Top 20 Most Active Publishers:")
print(top_publishers)

# Visualizations
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Bar chart of top publishers
top_publishers.plot(kind='barh', ax=axes[0])
axes[0].set_title('Top 20 Publishers by Article Count')
```

```

axes[0].set_xlabel('Number of Articles')
axes[0].invert_yaxis()

# Cumulative distribution
cumulative_pct = (publisher_counts.cumsum() / publisher_counts.sum() * 100)
axes[1].plot(range(len(cumulative_pct)), cumulative_pct)
axes[1].set_title('Cumulative Distribution of Articles by Publisher')
axes[1].set_xlabel('Publisher Rank')
axes[1].set_ylabel('Cumulative Percentage (%)')
axes[1].axhline(y=80, color='r', linestyle='--', label='80% threshold')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('notebooks/publisher_activity.png', dpi=300, bbox_inches='tight')
plt.show()

# Statistics
print(f"\nTotal unique publishers: {df['publisher'].nunique()}")
print(f"Publishers with >1000 articles: {(publisher_counts > 1000).sum()}")
print(f"Publishers with >100 articles: {(publisher_counts > 100).sum()}")
print(f"Publishers with only 1 article: {(publisher_counts == 1).sum()}")

```

Key Questions to Answer:

- Which publishers dominate the news feed?

1.3 Publication Date Trends

Objective: Analyze publication dates to identify trends over time.

Implementation Steps:

```

# Convert date column to datetime
df['date'] = pd.to_datetime(df['date'])

# Extract temporal features
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['day_of_week'] = df['date'].dt.day_name()
df['hour'] = df['date'].dt.hour
df['date_only'] = df['date'].dt.date

# Articles per year
articles_per_year = df.groupby('year').size()
print("Articles per Year:")
print(articles_per_year)

# Articles per month (across all years)

```

```

articles_per_month = df.groupby('month').size()
print("\nArticles per Month (Average):")
print(articles_per_month)

# Articles per day of week
articles_per_dow = df.groupby('day_of_week').size()
print("\nArticles per Day of Week:")
print(articles_per_dow)

# Visualizations
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Time series: Articles per day
daily_counts = df.groupby('date_only').size()
axes[0, 0].plot(daily_counts.index, daily_counts.values, linewidth=0.5)
axes[0, 0].set_title('Daily Article Publication Frequency')
axes[0, 0].set_xlabel('Date')
axes[0, 0].set_ylabel('Number of Articles')
axes[0, 0].tick_params(axis='x', rotation=45)
axes[0, 0].grid(True, alpha=0.3)

# Articles per month
articles_per_month.plot(kind='bar', ax=axes[0, 1], color='steelblue')
axes[0, 1].set_title('Articles per Month')
axes[0, 1].set_xlabel('Month')
axes[0, 1].set_ylabel('Number of Articles')
axes[0, 1].set_xticklabels(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                           'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])

# Articles per day of week
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sun
articles_per_dow = articles_per_dow.reindex(day_order)
articles_per_dow.plot(kind='bar', ax=axes[1, 0], color='coral')
axes[1, 0].set_title('Articles per Day of Week')
axes[1, 0].set_xlabel('Day of Week')
axes[1, 0].set_ylabel('Number of Articles')
axes[1, 0].tick_params(axis='x', rotation=45)

# Articles per year
articles_per_year.plot(kind='bar', ax=axes[1, 1], color='green')
axes[1, 1].set_title('Articles per Year')
axes[1, 1].set_xlabel('Year')
axes[1, 1].set_ylabel('Number of Articles')

plt.tight_layout()
plt.savefig('notebooks/publication_trends.png', dpi=300, bbox_inches='tight')
plt.show()

# Identify spikes (days with unusually high article counts)

```

```
mean_daily = daily_counts.mean()
std_daily = daily_counts.std()
spike_threshold = mean_daily + 2 * std_daily
spike_days = daily_counts[daily_counts > spike_threshold]

print(f"\nDays with spike in publications (>2 std dev): {len(spike_days)}")
print("Top 10 days with most articles:")
print(spike_days.nlargest(10))
```

Key Questions to Answer:

- Are there specific days of the week with more news?

2. Text Analysis (Topic Modeling)

2.1 Keyword and Phrase Extraction

Objective: Identify common keywords/phrases and extract significant events.

Implementation Steps:

```
import re
from collections import Counter
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk

# Download required NLTK data
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)

# Combine all headlines
all_headlines = ' '.join(df['headline'].astype(str))

# Convert to lowercase and tokenize
tokens = word_tokenize(all_headlines.lower())

# Remove stopwords and non-alphabetic tokens
stop_words = set(stopwords.words('english'))
# Add financial domain-specific stopwords
financial_stopwords = {'stock', 'stocks', 'shares', 'company', 'companies',
                      'market', 'markets', 'price', 'prices', 'trading'}
stop_words.update(financial_stopwords)

filtered_tokens = [token for token in tokens
                   if token.isalpha() and token not in stop_words and len(token) > 2]

# Count word frequencies
word_freq = Counter(filtered_tokens)
```

```

top_keywords = word_freq.most_common(50)

print("Top 50 Keywords:")
for word, count in top_keywords:
    print(f"{word}: {count}")

# Visualize top keywords
top_20_words = dict(top_keywords[:20])
plt.figure(figsize=(12, 8))
plt.barh(range(len(top_20_words)), list(top_20_words.values()))
plt.yticks(range(len(top_20_words)), list(top_20_words.keys()))
plt.xlabel('Frequency')
plt.title('Top 20 Keywords in Headlines')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig('notebooks/top_keywords.png', dpi=300, bbox_inches='tight')
plt.show()

```

2.2 Significant Event Detection

Objective: Extract significant events like "FDA approval", "price target", etc.

Implementation Steps:

```

# Define patterns for significant events
event_patterns = {
    'FDA Approval': r'fda\s+approval|approved\s+by\s+fda',
    'Price Target': r'price\s+target|target\s+price|raises\s+price\s+target|lowers\s+
    'Earnings': r'earnings|eps|q[1-4]\s+earnings|beats\s+estimate|misses\s+estimate',
    'Merger/Acquisition': r'merger|acquisition|acquires|acquired|takeover',
    'IPO': r'ipo|initial\s+public\s+offering|goes\s+public',
    'Analyst Rating': r'upgrade|downgrade|maintains|buy|sell|hold|outperform|underper
    'Stock Split': r'stock\s+split|split\s+announcement',
    'Dividend': r'dividend|dividend\s+announcement|dividend\s+increase',
    '52-Week High/Low': r'52-week\s+high|52-week\s+low|hits\s+52-week',
    'Bankruptcy': r'bankruptcy|files\s+for\s+bankruptcy|chapter\s+\d+'
}

# Create event detection columns
for event_name, pattern in event_patterns.items():
    df[f'event_{event_name.lower().replace("/", "_").replace(" ", "_")}'] = \
        df['headline'].str.contains(pattern, case=False, na=False, regex=True)

# Count occurrences of each event type
event_counts = {}
for event_name in event_patterns.keys():
    col_name = f'event_{event_name.lower().replace("/", "_").replace(" ", "_')}'
    event_counts[event_name] = df[col_name].sum()

print("\nSignificant Event Counts:")

```

```

for event, count in sorted(event_counts.items(), key=lambda x: x[1], reverse=True):
    print(f"{event}: {count}")

# Visualize event frequencies
plt.figure(figsize=(12, 6))
events_df = pd.DataFrame(list(event_counts.items()), columns=['Event', 'Count'])
events_df = events_df.sort_values('Count', ascending=False)
plt.barh(events_df['Event'], events_df['Count'])
plt.xlabel('Number of Articles')
plt.title('Frequency of Significant Events in Headlines')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig('notebooks/event_frequencies.png', dpi=300, bbox_inches='tight')
plt.show()

```

2.3 Topic Modeling

Objective: Use NLP to identify topics in the headlines.

Implementation Steps:

```

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.decomposition import NMF

# Sample data for topic modeling (use sample if dataset is too large)
sample_size = min(50000, len(df))
df_sample = df.sample(n=sample_size, random_state=42)

# Prepare text data
headlines = df_sample['headline'].astype(str).tolist()

# Create TF-IDF vectorizer
vectorizer = TfidfVectorizer(
    max_features=1000,
    stop_words='english',
    ngram_range=(1, 2), # Include unigrams and bigrams
    min_df=5, # Word must appear in at least 5 documents
    max_df=0.7 # Word must appear in less than 70% of documents
)

# Fit and transform
tfidf_matrix = vectorizer.fit_transform(headlines)

# Apply LDA (Latent Dirichlet Allocation)
n_topics = 10
lda = LatentDirichletAllocation(n_components=n_topics, random_state=42, max_iter=10)
lda.fit(tfidf_matrix)

# Display topics

```

```

feature_names = vectorizer.get_feature_names_out()

print("\nIdentified Topics (LDA):")
for topic_idx, topic in enumerate(lda.components_):
    top_words_idx = topic.argsort()[-10:][::-1]
    top_words = [feature_names[i] for i in top_words_idx]
    print(f"\nTopic {topic_idx + 1}: {' '.join(top_words)}")

# Alternative: NMF (Non-negative Matrix Factorization)
nmf = NMF(n_components=n_topics, random_state=42, max_iter=200)
nmf.fit(tfidf_matrix)

print("\n\nIdentified Topics (NMF):")
for topic_idx, topic in enumerate(nmf.components_):
    top_words_idx = topic.argsort()[-10:][::-1]
    top_words = [feature_names[i] for i in top_words_idx]
    print(f"\nTopic {topic_idx + 1}: {' '.join(top_words)}")

```

Key Questions to Answer:

- What are the main themes in financial news?

3. Time Series Analysis

3.1 Publication Frequency Over Time

Objective: Analyze how publication frequency varies over time and identify spikes.

Implementation Steps:

```

# Set date as index for time series analysis
df_time = df.set_index('date').sort_index()

# Resample by different time periods
hourly_counts = df_time.resample('H').size()
daily_counts = df_time.resample('D').size()
weekly_counts = df_time.resample('W').size()
monthly_counts = df_time.resample('M').size()

# Visualizations
fig, axes = plt.subplots(2, 2, figsize=(18, 12))

# Hourly pattern (aggregate across all days)
hourly_pattern = df_time.groupby(df_time.index.hour).size()
axes[0, 0].plot(hourly_pattern.index, hourly_pattern.values, marker='o')
axes[0, 0].set_title('Average Articles per Hour of Day')
axes[0, 0].set_xlabel('Hour of Day (UTC-4)')
axes[0, 0].set_ylabel('Number of Articles')
axes[0, 0].set_xticks(range(24))

```



```

axes[0, 0].grid(True, alpha=0.3)

# Daily counts over time
axes[0, 1].plot(daily_counts.index, daily_counts.values, linewidth=0.8)
axes[0, 1].set_title('Daily Article Publication Frequency')
axes[0, 1].set_xlabel('Date')
axes[0, 1].set_ylabel('Number of Articles')
axes[0, 1].tick_params(axis='x', rotation=45)
axes[0, 1].grid(True, alpha=0.3)

# Weekly counts
axes[1, 0].plot(weekly_counts.index, weekly_counts.values, linewidth=1.5)
axes[1, 0].set_title('Weekly Article Publication Frequency')
axes[1, 0].set_xlabel('Week')
axes[1, 0].set_ylabel('Number of Articles')
axes[1, 0].tick_params(axis='x', rotation=45)
axes[1, 0].grid(True, alpha=0.3)

# Monthly counts
axes[1, 1].plot(monthly_counts.index, monthly_counts.values, marker='o', linewidth=2)
axes[1, 1].set_title('Monthly Article Publication Frequency')
axes[1, 1].set_xlabel('Month')
axes[1, 1].set_ylabel('Number of Articles')
axes[1, 1].tick_params(axis='x', rotation=45)
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('notebooks/time_series_analysis.png', dpi=300, bbox_inches='tight')
plt.show()

# Identify significant spikes
def identify_spikes(series, threshold_std=2):
    mean = series.mean()
    std = series.std()
    threshold = mean + threshold_std * std
    spikes = series[series > threshold]
    return spikes

daily_spikes = identify_spikes(daily_counts, threshold_std=2)
print(f"\nDays with publication spikes (>2 std dev): {len(daily_spikes)}")
print("Top 10 spike days:")
print(daily_spikes.nlargest(10))

```

3.2 Publishing Time Analysis

Objective: Identify specific times when most news is released.

Implementation Steps:

```
# Extract time components
```

```

df['hour'] = df['date'].dt.hour
df['minute'] = df['date'].dt.minute
df['time_of_day'] = pd.cut(df['hour'],
                           bins=[0, 6, 12, 18, 24],
                           labels=['Night (0-6)', 'Morning (6-12)',
                                   'Afternoon (12-18)', 'Evening (18-24)'])

# Articles by time of day
time_distribution = df['time_of_day'].value_counts()
print("\nArticles by Time of Day:")
print(time_distribution)

# Hourly distribution
hourly_dist = df['hour'].value_counts().sort_index()

# Visualizations
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Hourly distribution (bar chart)
axes[0, 0].bar(hourly_dist.index, hourly_dist.values, color='steelblue')
axes[0, 0].set_title('Article Publication by Hour of Day')
axes[0, 0].set_xlabel('Hour (UTC-4)')
axes[0, 0].set_ylabel('Number of Articles')
axes[0, 0].set_xticks(range(24))
axes[0, 0].grid(True, alpha=0.3, axis='y')

# Time of day pie chart
time_distribution.plot(kind='pie', ax=axes[0, 1], autopct='%1.1f%%')
axes[0, 1].set_title('Distribution by Time of Day')
axes[0, 1].set_ylabel('')

# Heatmap: Day of week vs Hour
pivot_table = df.pivot_table(values='headline',
                              index='day_of_week',
                              columns='hour',
                              aggfunc='count',
                              fill_value=0)

day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sun
pivot_table = pivot_table.reindex(day_order)
sns.heatmap(pivot_table, ax=axes[1, 0], cmap='YlOrRd', cbar_kws={'label': 'Article Co
axes[1, 0].set_title('Heatmap: Day of Week vs Hour')
axes[1, 0].set_xlabel('Hour of Day')
axes[1, 0].set_ylabel('Day of Week')

# Peak hours identification
peak_hours = hourly_dist.nlargest(5)
axes[1, 1].barh(range(len(peak_hours)), peak_hours.values, color='coral')
axes[1, 1].set_yticks(range(len(peak_hours)))

```

```

axes[1, 1].set_yticklabels([f"Hour {h}" for h in peak_hours.index])
axes[1, 1].set_xlabel('Number of Articles')
axes[1, 1].set_title('Top 5 Peak Publication Hours')
axes[1, 1].invert_yaxis()

plt.tight_layout()
plt.savefig('notebooks/publishing_time_analysis.png', dpi=300, bbox_inches='tight')
plt.show()

print(f"\nPeak publication hour: {hourly_dist.idxmax()} (UTC-4)")
print(f"Articles during peak hour: {hourly_dist.max()}")
print(f"\nTop 5 hours with most articles:")
for hour, count in peak_hours.items():
    print(f"  Hour {hour}: {count} articles")

```

Key Questions to Answer:

- What times of day see the most news releases?

4. Publisher Analysis

4.1 Top Contributing Publishers

Objective: Identify which publishers contribute most to the news feed.

Implementation Steps:

```

# Publisher statistics
publisher_stats = df.groupby('publisher').agg({
    'headline': 'count',
    'stock': 'nunique',
    'date': ['min', 'max']
}).reset_index()

publisher_stats.columns = ['publisher', 'article_count', 'unique_stocks', 'first_arti
publisher_stats['date_range_days'] = (publisher_stats['last_article'] - publisher_sta
publisher_stats['articles_per_day'] = publisher_stats['article_count'] / (publisher_s

# Sort by article count
publisher_stats = publisher_stats.sort_values('article_count', ascending=False)

print("Top 20 Publishers by Article Count:")
print(publisher_stats.head(20)[['publisher', 'article_count', 'unique_stocks', 'artic

# Visualizations
fig, axes = plt.subplots(2, 2, figsize=(18, 12))

# Top 15 publishers by count
top_15 = publisher_stats.head(15)

```

```

axes[0, 0].barh(range(len(top_15)), top_15['article_count'].values)
axes[0, 0].set_yticks(range(len(top_15)))
axes[0, 0].set_yticklabels(top_15['publisher'].values, fontsize=8)
axes[0, 0].set_xlabel('Number of Articles')
axes[0, 0].set_title('Top 15 Publishers by Article Count')
axes[0, 0].invert_yaxis()

# Articles per day for top publishers
top_20 = publisher_stats.head(20)
axes[0, 1].barh(range(len(top_20)), top_20['articles_per_day'].values, color='orange')
axes[0, 1].set_yticks(range(len(top_20)))
axes[0, 1].set_yticklabels(top_20['publisher'].values, fontsize=8)
axes[0, 1].set_xlabel('Articles per Day')
axes[0, 1].set_title('Top 20 Publishers: Articles per Day')
axes[0, 1].invert_yaxis()

# Unique stocks covered by top publishers
axes[1, 0].scatter(top_20['article_count'], top_20['unique_stocks'], alpha=0.6)
axes[1, 0].set_xlabel('Total Articles')
axes[1, 0].set_ylabel('Unique Stocks Covered')
axes[1, 0].set_title('Publisher Coverage: Articles vs Stocks Covered')
axes[1, 0].grid(True, alpha=0.3)

# Distribution of article counts (log scale)
axes[1, 1].hist(publisher_stats['article_count'], bins=50, edgecolor='black', log=True)
axes[1, 1].set_xlabel('Article Count')
axes[1, 1].set_ylabel('Number of Publishers (log scale)')
axes[1, 1].set_title('Distribution of Publisher Activity (Log Scale)')
axes[1, 1].axvline(publisher_stats['article_count'].median(),
                  color='r', linestyle='--', label=f'Median: {publisher_stats["article_count"].median()}')
axes[1, 1].legend()

plt.tight_layout()
plt.savefig('notebooks/publisher_analysis.png', dpi=300, bbox_inches='tight')
plt.show()

```

4.2 News Type Differences by Publisher

Objective: Analyze if different publishers report different types of news.

Implementation Steps:

```

# Analyze event types by publisher
top_publishers_list = publisher_stats.head(10)['publisher'].tolist()

# Create event summary for top publishers
publisher_event_summary = []
for pub in top_publishers_list:
    pub_df = df[df['publisher'] == pub]
    pub_events = {}

```

```

for event_name in event_patterns.keys():
    col_name = f'event_{event_name.lower().replace("/", "_").replace(" ", "_')}'
    pub_events[event_name] = pub_df[col_name].sum()
pub_events['publisher'] = pub
pub_events['total_articles'] = len(pub_df)
publisher_event_summary.append(pub_events)

publisher_events_df = pd.DataFrame(publisher_event_summary)

# Calculate event percentages
event_cols = [col for col in publisher_events_df.columns if col not in ['publisher',
for col in event_cols:
    publisher_events_df[f'{col}_pct'] = (publisher_events_df[col] / publisher_events_

# Visualize event distribution by publisher
fig, axes = plt.subplots(figsize=(14, 8))
publisher_events_df.set_index('publisher')[event_cols].plot(kind='bar', stacked=True,
axes.set_title('Event Type Distribution by Top 10 Publishers')
axes.set_xlabel('Publisher')
axes.set_ylabel('Number of Articles')
axes.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
axes.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.savefig('notebooks/publisher_event_distribution.png', dpi=300, bbox_inches='tight
plt.show()

# Heatmap of event percentages
pct_cols = [col for col in publisher_events_df.columns if col.endswith('_pct')]
pct_df = publisher_events_df.set_index('publisher')[pct_cols]
pct_df.columns = [col.replace('_pct', '') for col in pct_df.columns]

plt.figure(figsize=(12, 8))
sns.heatmap(pct_df, annot=True, fmt='.1f', cmap='YlOrRd', cbar_kws={'label': 'Percent
plt.title('Event Type Percentage by Publisher (Top 10)')
plt.xlabel('Event Type')
plt.ylabel('Publisher')
plt.tight_layout()
plt.savefig('notebooks/publisher_event_heatmap.png', dpi=300, bbox_inches='tight')
plt.show()

```

4.3 Email Domain Analysis

Objective: If email addresses are used as publisher names, identify unique domains.

Implementation Steps:

```

import re

# Function to extract email domain
def extract_domain(publisher):

```

```

# Check if publisher is an email address
email_pattern = r'([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})'
match = re.search(email_pattern, str(publisher))
if match:
    return match.group(1).split('@')[1]
return None

# Extract domains
df['publisher_domain'] = df['publisher'].apply(extract_domain)

# Count articles by domain
domain_counts = df[df['publisher_domain'].notna()]['publisher_domain'].value_counts()

print(f"\nTotal publishers with email addresses: {df['publisher_domain'].notna().sum()}")
print(f"Unique email domains: {df['publisher_domain'].nunique()}")
print("\nTop 20 Email Domains:")
print(domain_counts.head(20))

# Visualizations
if len(domain_counts) > 0:
    fig, axes = plt.subplots(1, 2, figsize=(16, 6))

    # Top domains
    top_domains = domain_counts.head(15)
    axes[0].barh(range(len(top_domains)), top_domains.values)
    axes[0].set_yticks(range(len(top_domains)))
    axes[0].set_yticklabels(top_domains.index, fontsize=9)
    axes[0].set_xlabel('Number of Articles')
    axes[0].set_title('Top 15 Email Domains by Article Count')
    axes[0].invert_yaxis()

    # Domain distribution
    axes[1].hist(domain_counts.values, bins=30, edgecolor='black', log=True)
    axes[1].set_xlabel('Articles per Domain')
    axes[1].set_ylabel('Number of Domains (log scale)')
    axes[1].set_title('Distribution of Articles per Email Domain')
    axes[1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('notebooks/email_domain_analysis.png', dpi=300, bbox_inches='tight')
    plt.show()

# Identify organizational patterns
print(f"\nMost common domain: {domain_counts.index[0]} ({domain_counts.iloc[0]} a")
print(f"Domains with >1000 articles: {(domain_counts > 1000).sum()}")
print(f"Domains with only 1 article: {(domain_counts == 1).sum()}")

```

Key Questions to Answer:

- Which publishers/domains are most influential?

5. Additional Analysis Recommendations

5.1 Stock-Specific Analysis

```
# Articles per stock
stock_counts = df['stock'].value_counts()
print("Top 10 Stocks by Article Count:")
print(stock_counts.head(10))

# Average headline length by stock
headline_length_by_stock = df.groupby('stock')['headline_length'].mean().sort_values(
print("\nStocks with Longest Average Headlines:")
print(headline_length_by_stock.head(10))
```

5.2 Publisher-Stock Relationships

```
# Which publishers cover which stocks?
publisher_stock_matrix = df.pivot_table(
    values='headline',
    index='publisher',
    columns='stock',
    aggfunc='count',
    fill_value=0
)

# Find publishers with most diverse coverage
publisher_diversity = publisher_stock_matrix.apply(lambda x: (x > 0).sum(), axis=1).s
print("Publishers with Most Diverse Stock Coverage:")
print(publisher_diversity.head(10))
```

5.3 Data Quality Checks

```
# Check for missing values
print("Missing Values:")
print(df.isnull().sum())

# Check for duplicates
print(f"\nDuplicate headlines: {df['headline'].duplicated().sum()}")
print(f"Duplicate URLs: {df['url'].duplicated().sum()}")

# Check date range
print(f"\nDate Range: {df['date'].min()} to {df['date'].max()}")
print(f"Total time span: {(df['date'].max() - df['date'].min()).days} days")
```

6. Best Practices for Large Dataset Handling

Since your dataset is large (>1.4M rows), consider these strategies:

1. Use Chunking for Initial Loading:

```
chunks = []
for chunk in pd.read_csv('data/Data/raw_analyst_ratings.csv', chunksize=chunk_size):
    chunks.append(chunk)
df = pd.concat(chunks, ignore_index=True)
```

chunk_si.

2. Sample for Exploratory Analysis:

```
df_sample = df.sample(n=100000, random_state=42)
```

Use ra

3. Use Efficient Data Types:

```
df['date'] = pd.to_datetime(df['date'])
df['stock'] = df['stock'].astype('category')
df['publisher'] = df['publisher'].astype('category')
```

Conver

4. Save Intermediate Results:

```
df.to_parquet('data/processed/raw_analyst_ratings_processed.parquet')
```

Save p

7. Summary Checklist

Use this checklist to track your EDA progress:

Descriptive Statistics

- Headline length statistics calculated

Text Analysis

- Common keywords extracted

Time Series Analysis

- Publication frequency over time analyzed

Publisher Analysis

- Top publishers identified
-

8. Next Steps

After completing the EDA:

1. Document Findings: Update INTERIM_REPORT.md with your findings
-

Good luck with your analysis!